

Multi-Core Sigmoid (MCS)

A sigmoid implied-variance base with zero-wing hats for WW and event smiles

Vol-Fitter Technical Notes, No. 03

Vol-Fitter

July 9, 2026

Abstract

This note documents the Vol-Fitter’s sigmoid family, the *Multi-Core Sigmoid* (MCS) model. The base is a six-parameter sigmoid implied-variance slice: a log-cosh convexity primitive giving an ATM level, an ATM skew, a convexity amplitude, and *asymmetric* put/call wing steepnesses, C^2 across its centre. On top of the base the model adds R signed *zero-wing hat* kernels — normalized centred second differences of the log-cosh primitive — each of which inserts a local variance hump ($\alpha > 0$) or notch ($\alpha < 0$) *without moving the base wing slopes*. This is exactly the structure a single hyperbola (SVI, Note 02) cannot provide: we prove that a globally convex slice cannot reproduce a WW-shaped smile (central trough with two shoulders), and show that two hats reduce the fit error on such a smile from 122.59 to 2.43 volatility basis points. The model carries $6 + 4R$ parameters, has a closed-form Durrleman butterfly diagnostic, and is calibrated in three stages (base $\rightarrow$ greedy hat seeding $\rightarrow$ joint bounded refine) with an analytic Jacobian measured $2.87\times$ faster than finite differences. The cores count R is the user’s “cores” slider — the direct analogue of the LQD Legendre order — capped at two, because the offline backtest shows deeper stacks over-fit and manufacture butterfly arbitrage disproportionately in the *put* wing; a default-on put-wing Durrleman regularizer that vanishes on any admissible slice, and a note on that over-fit tendency, close the discussion.

Contents

1	Motivation: what a hyperbola cannot do	2
2	The sigmoid base	3
2.1	Coordinates and the log-cosh primitive	3
2.2	The six base parameters	3
3	Zero-wing hats	3
3.1	Definition	3
3.2	The full model	4
4	Static no-arbitrage	4
4.1	Where the arbitrage lives, and the put-wing regularizer	5
5	Calibration	5
6	Worked example: a WW smile	6

7	Original ideas, and the over-fit caution	8
8	Traceability	8
A	Hyperparameter atlas	9
B	Performance notes	10
C	Reference implementation	10

1 Motivation: what a hyperbola cannot do

SVI's strength — one convex hyperbola — is also its ceiling. Some smiles are not convex. Ahead of a binary event, or in names with bimodal positioning, the implied-volatility curve develops a *WW* shape: a trough near the money, a *shoulder* (local maximum) on each side, and then wings that turn up again toward the Lee asymptotes. Figure 1 shows such a target; the base sigmoid alone (the dotted curve) misses the shoulders by 122.59 vol bps.

Proposition 1 (A convex slice cannot make a WW). *If total variance $w(k)$ is convex on an interval, then on that interval w has no interior strict local maximum, hence no shoulder flanked by two troughs. A WW smile, which has interior local maxima, therefore cannot be represented by any globally convex slice (in particular by raw SVI, whose $w'' > 0$ everywhere).*

Proof. A convex function on an interval lies below the chord between any two points and its one-sided derivatives are non-decreasing; an interior strict local maximum would force the derivative to change from positive to negative, contradicting monotone non-decreasing slope. SVI has $w''(k) = b\sigma^2/r^3 > 0$ (Note 02), so it is strictly convex. $\square$

The MCS answer is to keep a convex *base* that owns the wings and the overall skew, and to add *local*, *wing-neutral* corrections that can be positive or negative.

Invariant

1. Adding cores never moves the asymptotic wings: every hat and its first two derivatives vanish in both tails (lemma 1).
2. Expressiveness is budgeted: cores are capped by the quote count (9) and the surfaced slider is clamped to two.
3. The model reports its own butterfly diagnostic g per fit, and the default-on put-wing hinge is exactly zero on any admissible slice (byte-identical, test-locked).
4. The parameters are a means, not the object: hats are non-unique by design, and only the *curve* is contractual (section 5).

The design constraint on the correction kernels is sharp:

Heuristic

A shoulder is a *local* feature: it changes the body of the smile in a bounded region but must not alter the asymptotic wings (those are pinned by Lee and by liquid risk-reversals). So the correction kernels must vanish — in value, slope, *and* curvature — in both tails. That “zero-wing” requirement is what singles out the centred-second-difference hat of section 3, and it is why an arbitrary Gaussian bump would be wrong (a Gaussian decays but tilts the local skew on the way out).

2 The sigmoid base

2.1 Coordinates and the log-cosh primitive

Work in the dimensionless log-strike

$$z = \frac{k}{\sigma_{\text{ref}}\sqrt{t}}, \quad (1)$$

where σ_{ref} is the near-the-money quoted volatility, so that z measures moneyness in standard deviations and the model is roughly scale-free across expiries. The convexity primitive is the *log-cosh*

$$\Phi_\kappa(u) = \frac{4}{\kappa^2} \log \cosh\left(\frac{\kappa u}{2}\right), \quad \Phi'_\kappa(u) = \frac{2}{\kappa} \tanh\left(\frac{\kappa u}{2}\right), \quad \Phi''_\kappa(u) = \text{sech}^2\left(\frac{\kappa u}{2}\right). \quad (2)$$

Φ_κ is a smooth, convex, even softened absolute value: $\Phi_\kappa(u) \sim \frac{2}{\kappa}|u| - \frac{4}{\kappa^2} \log 2$ for large $|u|$ (a wing of slope $2/\kappa$) and $\Phi_\kappa(u) \approx u^2/2$ near 0 (the rounded vertex), with κ the transition steepness. For numerical safety the log-cosh is replaced by its asymptote $|x| - \log 2$ beyond $|x| = 50$ to avoid cosh overflow.

2.2 The six base parameters

The base slice and its z -derivatives are

$$\begin{aligned} v_{\text{base}}(z) &= V_0 + S_0(z - z_0) + K_0 \Phi_{\kappa_i}(z - z_0), \\ v'_{\text{base}}(z) &= S_0 + K_0 \Phi'_{\kappa_i}(z - z_0), \quad v''_{\text{base}}(z) = K_0 \Phi''_{\kappa_i}(z - z_0), \end{aligned} \quad (3)$$

with the steepness switching $\kappa_i = \kappa_P$ for $z < z_0$ and $\kappa_i = \kappa_C$ for $z \geq z_0$. The six parameters are the variance level V_0 at the centre z_0 , the skew S_0 , the convexity amplitude $K_0 \geq 0$, the centre z_0 , and the *asymmetric* put/call wing steepnesses κ_P, κ_C . Because Φ and Φ' both vanish at the origin, the slice is C^2 across z_0 even though the steepness switches there — so the asymmetry is in the *rate* at which each wing straightens, not a kink. The asymptotic z -space variance wing slopes are

$$v'_{\text{base}}(z) \rightarrow S_0 \mp \frac{2K_0}{\kappa_{P,C}} \quad (z \rightarrow \mp\infty), \quad (4)$$

i.e. a left slope $S_0 - 2K_0/\kappa_P$ and a right slope $S_0 + 2K_0/\kappa_C$. These are the only handles that set the wings, a fact section 3 exploits.

3 Zero-wing hats

3.1 Definition

The correction kernel is the normalized centred second difference of the log-cosh primitive,

$$B_{c,h,\kappa}(z) = \frac{\Phi_\kappa(u-h) - 2\Phi_\kappa(u) + \Phi_\kappa(u+h)}{2\Phi_\kappa(h)}, \quad u = z - c, \quad (5)$$

centred at c , of half-width h and steepness κ , normalized so that $B_{c,h,\kappa}(c) = 1$. Its derivatives are the same second difference of Φ' and Φ'' over the same normalizer.

Lemma 1 (Zero wings). $B_{c,h,\kappa}(z) \rightarrow 0$, $B'_{c,h,\kappa}(z) \rightarrow 0$ and $B''_{c,h,\kappa}(z) \rightarrow 0$ as $z \rightarrow \pm\infty$.

Proof. For large $|u|$, $\Phi_\kappa(u) = \frac{2}{\kappa}|u| - \frac{4}{\kappa^2} \log 2 + o(1)$ is asymptotically *affine*, so the centred second difference $\Phi_\kappa(u-h) - 2\Phi_\kappa(u) + \Phi_\kappa(u+h) \rightarrow 0$ (a second difference annihilates affine functions). The same holds for Φ' (which tends to the constants $\pm 2/\kappa$) and Φ'' (which decays). Hence B and its first two derivatives vanish in both tails. $\square$

Lemma 1 is the whole point: adding αB to the base changes the body but leaves the wing slopes (4) *exactly* unchanged. The amplitude α is in variance units and, since $B(c) = 1$, is approximately the variance displacement at the centre. A positive α raises a shoulder; a negative α digs a notch. The curvature at the centre is $B''(c) = 2(\text{sech}^2(\kappa h/2) - 1)/(2\Phi_\kappa(h)) < 0$, so a positive hat is locally concave (a hump), as it should be.

The two objects read directly as (2)–(5):

Listing 1: The primitive Φ_κ and the zero-wing hat $B_{c,h,\kappa}$ (distilled from `models/sigmoid/kernels.py`; agrees with it to 10^{-13}).

```

1  def phi(u, kappa):                                # log-cosh primitive
    Phi_kappa
2      x = np.clip(kappa * u / 2, -50, 50)           # guard cosh() overflow
3      return (4 / kappa**2) * np.log(np.cosh(x))
4
5  def hat(z, c, h, kappa):                           # zero-wing kernel B_{c,h,
    kappa}
6      u = z - c                                     # centred 2nd difference
    of Phi:
7      raw = phi(u-h, kappa) - 2*phi(u, kappa) + phi(u+h, kappa)
8      return raw / (2 * phi(h, kappa))              # normalized so B(c) = 1

```

3.2 The full model

The Multi-Core Sigmoid (MCS) slice is the base plus R signed hats,

$$v_R(z) = v_{\text{base}}(z) + \sum_{r=1}^R \alpha_r B_{c_r, h_r, \kappa_r}(z), \quad w(k) = t v_R(z), \quad (6)$$

with $6 + 4R$ parameters. By lemma 1 every member of this family has the *same* Lee-compatible wings as its base, regardless of R — so adding expressiveness in the body never threatens the tails. The cores count R is the “cores” slider, the analogue of the LQD Legendre order N and the SVI vs LQD vs LV expressiveness dial.

4 Static no-arbitrage

Butterfly-freedom is the Durrleman condition (7), evaluated by converting the z -space variance and its derivatives to total-variance k -space derivatives via $k = \sigma_{\text{ref}} \sqrt{t} z$ and $w = t v$:

$$g(k) = \left(1 - \frac{k w'}{2w}\right)^2 - \frac{w'^2}{4} \left(\frac{1}{w} + \frac{1}{4}\right) + \frac{w''}{2} \geq 0. \quad (7)$$

Since the risk-neutral density is proportional to g , the diagnostic $g(k) \geq 0$ on the grid is the no-butterfly check. Unlike LQD, MCS is *not* arbitrage-free by construction: a large or narrow hat

can drive g negative, so g is reported per fit, the calibration bounds (section 5) keep hats in a benign range, and a default-on *put-wing regularizer* (section 4.1) actively penalizes any residual violation. The wings are Lee-compatible by construction (lemma 1); the cross-model wing treatment is Note 09.

4.1 Where the arbitrage lives, and the put-wing regularizer

When cores over-reach, they break (7) not near the money — where quotes are dense and discipline the curvature — but out in the *sparingly quoted wings*, where each hat’s local curvature is unconstrained by data. The offline backtest (project `backtest/`) localizes the violation sharply: across refit multi-core nodes about 64% of the $g < 0$ points sit in the *put* wing (the steep, variance-loaded side of the equity skew) against only $\sim 4\%$ near the money, with the worst point at a median standardized moneyiness $z \approx -3.2$. To discipline exactly that region, the joint refine stage carries an optional soft *put-wing Durrleman penalty*. On a grid $\{z_j\}_{j=1}^M$ of $M = 49$ points spanning a padding $\Delta z = 2$ in z *past* the quoted range (so it reaches into the unquoted tails), it appends the residual rows

$$r_j^{\text{wing}} = \sqrt{\lambda_j} \max(-g(z_j), 0), \quad \lambda_j = \Lambda \lambda_0 (1 + \mathbf{1}\{z_j < 0\}), \quad (8)$$

with g the model’s analytic Durrleman functional (7) and $\lambda_0 = 10^3$ the base strength. The surfaced dial is $\Lambda = \text{SivWingPenaltyPct}/100$ (default 1; 0 disables), and the indicator *doubles* the weight on the put side to match the measured asymmetry. Because $\max(-g, 0) = 0$ wherever the slice is butterfly-free, (8) is *exactly zero on an admissible fit*: a liquid, arbitrage-free name is byte-identical with or without it. The penalty bites only where a hat would otherwise manufacture a wing violation, pulling the fit back to the admissible boundary there rather than everywhere.

Heuristic

The penalty is a *one-sided* hinge on the same g the model already reports, sampled where the model extrapolates rather than where it interpolates. It does not *guarantee* $g \geq 0$ (a soft penalty trades the violation against the data fit), but on genuinely arbitrated illiquid wings it shrinks the worst violation by orders of magnitude — the backtest measured a median wing $\min g$ moving from ≈ -7.9 to ≈ -0.02 — at the cost of fitting *less* to the arbitrated de-Americanized quotes that caused the violation, which is the right trade (Note 05 removes that arbitrage at the source; section 7).

5 Calibration

The fit is a three-stage workflow that respects the model’s layered structure.

1. **Base fit** ($R = 0$). Fit the six base parameters to the mid quotes under bound constraints, from a data-driven start that reads the ATM variance, the local skew and the local convexity off three interpolated points. This stage is always mid, so its residual is meaningful for the next stage.
2. **Greedy hat seeding**. Compute the base residual in variance space and place each of the R hats at the largest remaining |residual|, masking a neighbourhood after each placement so two hats cannot seed on the same feature. Each seed starts at half-width $h = 0.40$ and steepness $\kappa = 5.0$, with amplitude initialized *from* the local residual and clipped to $[-1, 1]$ variance units.

3. **Joint bounded refine.** Refine all $6+4R$ parameters together under the requested objective (mid or band) with a mild ridge $\sqrt{\varrho}\alpha_r$ on the hat amplitudes, $\varrho = 0.01$. The ridge keeps overlapping cores from exploding against each other without biasing a well-determined amplitude.

All positive parameters (K_0 , the steepnesses, the hat half-widths and steepnesses) are bound-constrained directly through scipy’s trust-region reflective solver; the hat half-width is held in $[0.15, 1.5]$ and the steepness in $[1.0, 12.0]$. Crucially the cores count is capped so the model never has more parameters than quotes,

$$R \leq \left\lfloor \frac{N_{\text{quotes}} - 6}{4} \right\rfloor, \quad (9)$$

which guards sparse short-dated chains against fitting spurious narrow kernels. Independently of this data-driven bound, the surfaced `nCores` is *hard-capped at two*, $R \in \{0, 1, 2\}$: the backtest found that $R \geq 3$ buys no out-of-sample precision while manufacturing the put-wing arbitrage of section 4.1, so a persisted desk setting above two is silently *clamped* (not rejected) on load. Precision matters here: the clamp lives in the *schema* (`FitSettings.nCores`), not in the calibrator — `calibrate_sigmoid(n_cores=3)` runs happily, and the library’s own tests exercise it. The cap is a product decision about what the app will persist and display, enforced at the settings boundary; research code below that boundary deliberately keeps the full family. Relatedly, the hats are *non-unique* by construction (two overlapping hats can trade amplitude against each other): the round-trip test recovers the *curve*, not the parameters, and nothing downstream ever consumes hat parameters directly. The refine stage carries the same optional blocks as the other models — band fit (Note 07), var-swap target (Note 08), the model-agnostic calendar hinge (Note 10), prior persistence (Note 13), and the put-wing regularizer (8) — so the sigmoid overlay is no longer a prior exception.

Performance

The dominant cost of an R -core fit is the $(6 + 4R + 1)$ -evaluation finite-difference Jacobian. The closed-form Jacobian of the residual stack (data + ridge + calendar) propagated through the log-cosh primitive replaces it in one pass; a fresh run on the two-core WW fit measured 70.1 ms vs 200.9 ms ($2.87\times$) with the optimum unchanged to $4.2e - 16$ in cost. It is gated to the var-swap/prior-free configuration, as in LQD and SVI. When the put-wing regularizer (8) is active the Jacobian is *hybrid*: analytic for the data/ridge/calendar rows and finite-difference only for the cheap 49-point g -penalty grid, so the penalized fit keeps essentially the analytic speed. See appendix B.

6 Worked example: a WW smile

The target in fig. 1 is a synthetic WW: rising V wings with a shoulder on each side of the ATM trough — the shape proposition 1 forbids a hyperbola — and it is itself *verified butterfly-clean* before any fit runs (see the remark closing this section). The sigmoid base ($R = 0$) fits it only to 122.59 vol bps, missing both shoulders. Two hats ($R = 2$, $6 + 4 \cdot 2 = 14$ parameters) reduce the maximum error to 2.43 vol bps while preserving the wing slopes $(S_0 - 2K_0/\kappa_P, S_0 + 2K_0/\kappa_C) = (-0.007, 0.007)$ and keeping the fit butterfly-free (section 6, $g(k) \geq 0$). (How many cores a WW needs depends on the target: this note’s WW resolves at $R = 2$, while the library’s own, sharper synthetic requires $R = 3$ — which the uncapped library test exercises, below the schema clamp of section 5.) Figure 2 shows the decomposition: the convex base owns the V, and the two signed hats add precisely the

two shoulders, each localized and wing-neutral.

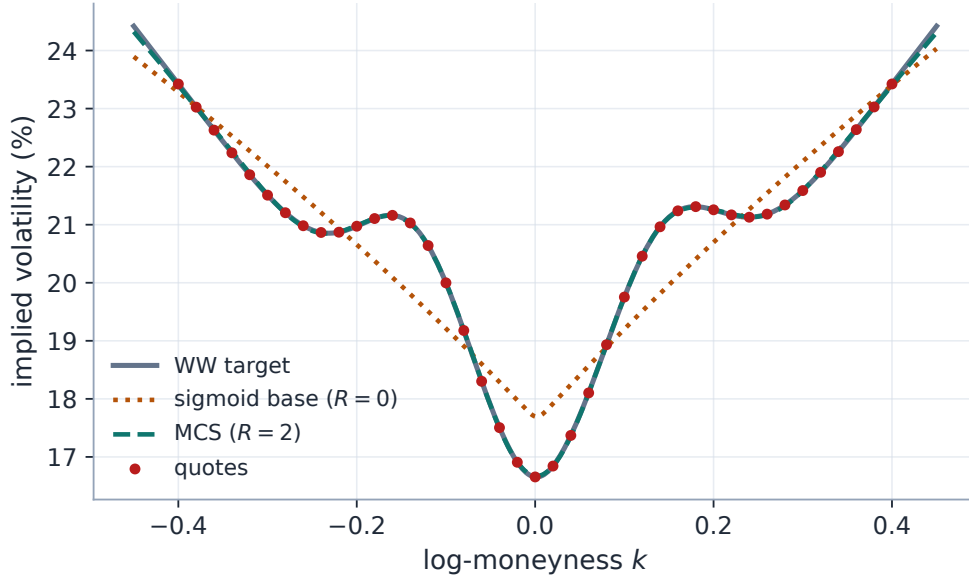


Figure 1: The WW smile: the convex base ($R = 0$, dotted) misses the shoulders by 122.59 vol bps; the two-core MCS fit (dashed) reaches 2.43 vol bps.

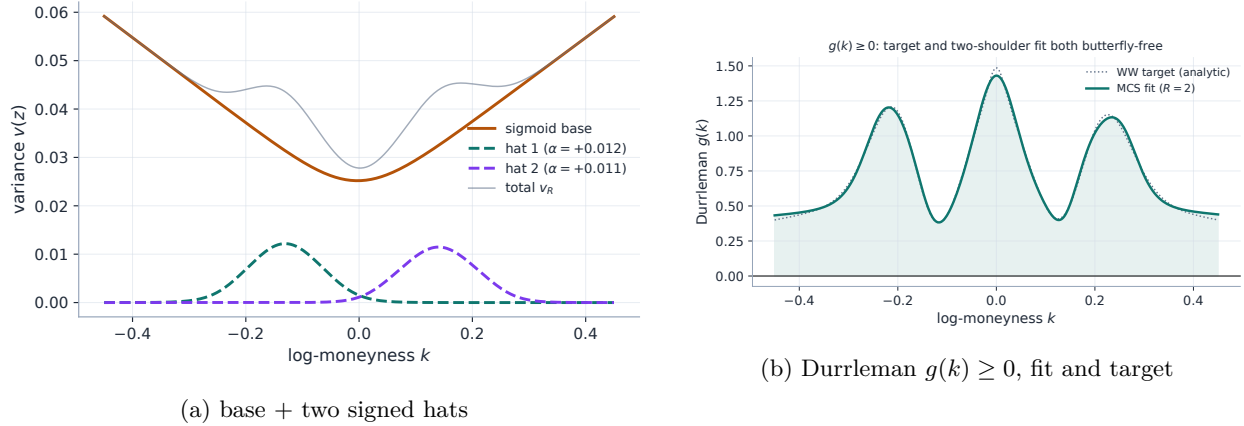


Figure 2: Left: the additive decomposition — the convex base owns the wings, the hats own the shoulders. Right: the butterfly diagnostic stays non-negative for *both* curves — the fitted slice and the synthetic target it chased, whose own analytic g is asserted positive by the generator.

Remark 1 (The target must be clean too — a fixed figure bug). A certificate is only as strong as the exercise it certifies. An earlier revision of this note’s generator built a WW target whose *own* Durrleman function dipped to $g \approx -0.38$ near the shoulders: the synthetic “market” itself carried butterfly arbitrage, so no admissible slice could both track it closely and honour $g \geq 0$, and the figure’s claim was quietly false. The fix re-chose the shoulder amplitudes and widths —

the shoulder-top curvature scales like $2A/s^2$ and is what drives w'' , hence g , negative — and the wing slope, so the target is genuinely admissible; the generator now computes the target’s g from *analytic* jets (no finite differences, the measurement lesson of Note 02) and asserts $g > 0$ on the plotted grid before any fit runs. Current values: target min $g = 0.38$, fitted slice min $g = 0.38$ (`figures/gen_siv.py`).

7 Original ideas, and the over-fit caution

The genuinely original elements are the *zero-wing hat* (5)–lemma 1 — a correction basis that is local in the body and provably neutral in the wings, so expressiveness and tail control are decoupled — and the *layered three-stage calibration* that places the hats on residuals rather than optimizing them blind. Together they let one model span from a clean index smile ($R = 0$, equivalent to a sigmoid SVI) to a bimodal event smile ($R = 2$) on a single slider.

Caution

With great expressiveness comes over-fitting. The offline backtest (project `backtest/`) found that on liquid index chains the Multi-Core hats *over-fit*: cores chase quote noise and manufacture butterfly-arbitrage (the analytic diagnostic flags $g < 0$ on a majority of calibrated nodes at $R \geq 2$ on dense data), while the base $R = 0$ behaves like SVI; the extra cores buy in-sample error and *give it back out-of-sample* — the marginal precision is negative for $R \geq 3$. Two measures followed. The amplitude ridge, the identifiability cap (9), and the g diagnostic mitigate the fit itself; the surfaced slider is *hard-capped at two* (section 5); and the put-wing regularizer (8) of section 4.1 is default-on to catch the residual wing violations that remain within the cap. The operational guidance is unchanged: *use cores only when the smile is genuinely non-convex* (events, bimodal positioning), and prefer LQD or Local-Vol for routine fitting. The cores slider is a scalpel, not a default.

Remark 2 (Two defences, one pathology — and they are complementary). The put-wing violation has *two* sources: the flexible MCS genuinely over-reaching (cured by (8)), and *arbitraged inputs* — the per-strike de-Americanization of Note 05 can hand every model a call curve that is already slightly non-convex in the wings. An ablation isolating the two defences (the de-Am convex repair of Note 05 versus the penalty (8)) on illiquid American ETFs found them *complementary, not redundant*. On the arb-prone population (medians over 38 nodes): with neither, worst-point $g \approx -30$ and in-sample RMS 92 bp; the de-Am repair alone cuts the violation about threefold *and lowers* the RMS to 25 bp (it removes the arbitrated quotes the model was chasing); the penalty alone nearly eliminates the violation ($g \geq -0.02$) but at 749 bp (it must fight those same quotes); both attain the penalty’s arbitrage removal at 225 bp, because the inputs are cleaned before the constraint is imposed. One real node makes the mechanism vivid (EFA in the Aug-2024 vol spike, 11 days to expiry, 22 quotes): neither defence, 75 bp with worst $g = -116$; repair alone, a tight 18 bp but $g = -12$ remains; penalty alone, arbitrage-free but 726 bp; both, arbitrage-free at 34 bp. The fence is expensive only when it must fight arbitrated inputs — the repair is what makes it affordable. On liquid names both are byte-identical no-ops. That is the quantitative case for shipping both on by default; the census that sited the penalty (64% put wing, median worst violation at $z \approx -3.2$) and the cross-model framing are Note 09’s case file and confinement principle.

8 Traceability

Table 1: Claims in this note and the code/tests that lock them.

Claim	Object	Code anchor	Test anchor
Hat is unit-height, flat at centre, lemma 1 zero-wing in value/slope/curvature		models/sigmoid/kernels.py	test_sigmoid.py:: test_hat_is_zero_wing_and_unit_height
Wing slopes preserved under (4) cores; g and min v diagnostics match note		models/sigmoid/sigmoid.py	test_sigmoid.py:: test_note_arbitrage_diagnostics
Cores improve the WW fit monotonically	section 6	models/sigmoid/calibrate.py	test_sigmoid.py:: test_more_cores_fit_better_on_ww_smile
Identifiability cap $6 + 4R \leq N$ (9)		models/sigmoid/calibrate.py	test_sigmoid.py:: test_cores_are_capped_to_quote_count
Surfaced slider clamped to two (schema boundary)	section 5	api/schemas.py	test_siv_wing_penalty.py::test_cores_clamped_to_two
Put-wing hinge removes wing arb; (8) byte-identical on clean slices; off by library default		models/sigmoid/calibrate.py	test_siv_wing_penalty.py::test_penalty_removes_wing_arb, ::test_penalty_byte_identical_on_arb_free_slice, ::test_penalty_off_default
Analytic Jacobian = FD (base, cores, band, calendar)	section 5	models/sigmoid/jacobian.py	test_sigmoid_jacobian.py::test_two_cores_mid, ::test_two_cores_band, ::test_calendar_active
Curve identified, parameters deliberately not	section 5	models/sigmoid/calibrate.py	test_sigmoid.py::test_round_trip_curve_recovery
Prior blocks reach the MCS overlay	section 5	models/sigmoid/calibrate.py	test_prior_parametric.py::test_operator_prior_pulls_all_models_toward_prior_skew
R3×R6 ablation numbers	section 7	backend/backtest/ablation_arb.py	test_ablation_arb.py
WW figure target and fit butterfly-clean ($g > 0$ asserted)	remark 1	figures/gen_siv.py	runtime assertion in the generator (no separate test)

A Hyperparameter atlas

Table 2: Multi-Core Sigmoid (MCS) hyperparameters.

Knob	Default	Role
<i>Surfaced (FitSettings)</i>		
nCores R	2	Number of zero-wing hats, $R \in \{0, 1, 2\}$ (schema-clamped at two — a persisted larger value is silently reduced; the library itself is uncapped, and the identifiability bound (9) may reduce R further). 0 = pure sigmoid base.
sivWingPenaltyPct Λ	100	Put-wing Durrleman penalty (8) strength; 100 = base $\lambda_0 = 10^3$, 0 = off.
sigmoidRidge ϱ	0.01	ℓ^2 penalty on hat amplitudes in the refine stage.
midAnchorWeight	0.05	Band-fit mid anchor (Note 07).
weightScheme	equal	Per-quote weights (Note 07).
calendarWeight	10^6	Calendar hinge penalty (Note 10; lives in OptionsSettings, not FitSettings).
<i>Internal (seeding / bounds)</i>		
_H_INIT	0.40	Hat half-width seed.
_KAPPA_INIT	5.0	Hat steepness seed.
_H_BOUNDS	[0.15, 1.5]	Hat half-width bounds.
_KAPPA_BOUNDS	[1.0, 12.0]	Hat steepness bounds.
_C_PAD	0.5	Centre padding beyond the quoted z -range.
_V_FLOOR	10^{-8}	Variance floor so $\sqrt{v}$ stays real.
_LOGCOSH_CLIP	50	$ x $ beyond which log cosh uses its asymptote.
xtol/ftol	10^{-12}	trf tolerances.

B Performance notes

1. **Analytic Jacobian** through the log-cosh primitive replaces the $(6 + 4R + 1)$ -evaluation finite difference; measured $2.87\times$ on the WW fit, optimum unchanged to $4.2e - 16$. Gated to the var-swap/prior-free path. When the put-wing penalty (8) is active, only its 49-point g -grid is finite-differenced (a *hybrid* Jacobian), so the penalized fit retains essentially this speed.
2. **Layered warm start.** Fitting the base first and seeding hats on its residual gives the joint refine an excellent start, so the bounded trf converges quickly even with several cores.
3. **Vectorized kernels.** Φ, Φ', Φ'' and the hat second differences are NumPy-vectorized and side-effect free, shared between the model and its Jacobian.
4. **Identifiability cap.** Limiting R by the quote count avoids wasting iterations on under-determined narrow kernels and is the first line of defence against the over-fit of section 7.

C Reference implementation

Building on listing 1, the full slice $v_R(z)$ (6) and the butterfly diagnostic $g(k)$ (7) are a few more lines. The z -jets (value, slope, curvature) are carried together because g needs all three; $_dhat$ forms a hat or either of its derivatives from the matching Φ -jet, so the zero-wing second difference is written once. Verified against MultiCoreSiv on a two-core benchmark to 10^{-13} .

Listing 2: The full model $v_R(z)$ and Durrleman $g(k)$ (distilled from `models/sgmoid/{kernels,sgmoid}.py`).

```
1 def phi_p(u, k): return (2/k) * np.tanh(k*u/2)           # Phi '  
2 def phi_pp(u, k): return 1 / np.cosh(k*u/2)**2          # Phi ''  
3 def _dhat(f, z, c, h, kappa):                             # B or a B-  
    derivative                                              #  
4     u = z - c                                           # from the  
    matching Phi-jet  
5     return (f(u-h,kappa) - 2*f(u,kappa) + f(u+h,kappa)) / (2*phi(h,  
        kappa))  
6  
7 def v_model(z, base, cores):  
8     """MCS variance  $v_R(z) = v_{base}(z) + \sum_r \alpha_r B_r(z)$ , with  $z$ -  
    jets."""  
9     v0, s0, k0, z0, kp, kc = base  
10    u = z - z0  
11    kappa = np.where(u < 0, kp, kc)                      # asymmetric  
    wings,  $C^2$  at  $z0$   
12    v, vz, vzz = v0 + s0*u + k0*phi(u,kappa), s0 + k0*phi_p(u,kappa),  
        k0*phi_pp(u,kappa)  
13    for alpha, c, h, kap in cores:                       # add the  
        signed zero-wing hats  
14        v += alpha * _dhat(phi, z, c, h, kap)  
15        vz += alpha * _dhat(phi_p, z, c, h, kap)  
16        vzz += alpha * _dhat(phi_pp, z, c, h, kap)  
17    return v, vz, vzz  
18  
19 def durrleman_g(z, base, cores, t, sigma_ref):  
20     """Butterfly diagnostic  $g(k) \geq 0$ ; converts  $z$ -space jets to  $k$ -  
    space."""  
21    v, vz, vzz = v_model(z, base, cores)  
22    k, w = sigma_ref*np.sqrt(t)*z, t*v  
23    wk, wkk = np.sqrt(t)/sigma_ref*vz, vzz/sigma_ref**2  
24    return (1 - k*wk/(2*w))**2 - 0.25*wkk**2*(1/w + 0.25) + 0.5*wkk
```

References

- [1] J. Gatheral. *The Volatility Surface: A Practitioner's Guide*. Wiley, 2006.
- [2] V. Durrleman. From implied to spot volatilities. *Finance and Stochastics*, 14(2):157–177, 2010.
- [3] R. W. Lee. The moment formula for implied volatility at extreme strikes. *Mathematical Finance*, 14(3):469–480, 2004.